AutoPDR: Circuit-Aware Solver Configuration Prediction for Hardware Model Checking

Guangyu Hu[1], Chen Chen[2], Xiaofeng Zhou[1], Jiaxi Zhang[3], Wei Zhang[1], Hongce Zhang[2,4]
[1]The Hong Kong University of Science and Technology
Emails: ghuae@connect.ust.hk, xzhoubu@connect.ust.hk, wei.zhang@ust.hk
[2]The Hong Kong University of Science and Technology (Guangzhou)
Emails: cchen099@connect.hkust-gz.edu.cn, hongcezh@hkust-gz.edu.cn
[3]Peking University
Email: zhangjiaxi@pku.edu.cn
[4]Corresponding author

arXiv:2603.25048v1 [cs.AR] 26 Mar 2026

***Abstract*—Property Directed Reachability (PDR) is a powerful algorithm for formal verification of hardware and software systems, but its performance is highly sensitive to parameter configurations. Manual parameter tuning is time-consuming and requires domain expertise, while traditional automated parameter tuning frameworks are not well-suited for time-sensitive verification tasks like PDR. This paper presents a circuit-aware solver configuration framework that employs graph learning for intelligent heuristic selection in PDR-based verification. Our approach combines graph representations with static circuit features to predict optimal PDR solving configurations for specific circuits. We incorporate expert prior knowledge through constraint-based parameter filtering to eliminate invalid and inefficient configurations and reduce 78% search space. Our feature extraction pipeline captures structural, functional, and connectivity characteristics of circuit topology and component patterns. Experimental evaluation on a comprehensive benchmark suite demonstrates significant performance improvements compared to default configurations and commonly-used settings. The system successfully identifies circuit-specific parameter patterns and automatically selects the most suitable solving strategies based on circuit characteristics, making it a practical tool for automated formal verification workflows.**

***Index Terms*—formal verification, property directed reachability, IC3, parameter tuning, machine learning, graph neural networks, automated reasoning, solver optimization.**

I. Introduction

Formal verification has become an essential component of modern hardware and software design, providing mathematical guarantees about system correctness. Among the various verification techniques, Property Directed Reachability (PDR), also known as IC3 [1], has emerged as one of the most effective algorithms for proving safety properties of finite-state systems. PDR's success stems from its ability to incrementally construct inductive invariants while avoiding the explicit enumeration of reachable states, making it particularly well-suited for large-scale verification problems [2], [3].

However, PDR's effectiveness is highly dependent on its parameter configuration. One of the commonly-used PDR implementations is built in `ABC` [3], which exposes numerous tunable parameters that control various aspects of its operation, including generalization strategies, abstraction mechanisms, proof obligation handling, and heuristic choices. These parameters significantly impact both the runtime performance and the likelihood of successful verification [4], [5]. Unfortunately, finding optimal parameter settings is a challenging task that typically requires extensive domain expertise and time-consuming manual experimentation [6]. Moreover, the combinatorial explosion of parameter combinations makes parallel exploration computationally infeasible, as it would quickly exhaust available computational resources, such as CPU cores and memory.

The parameter tuning problem in PDR is particularly complex due to several factors. First, the optimal parameter configuration is highly dependent on the structural characteristics of the circuit being verified. Parameters that work well for control-intensive designs may perform poorly on datapath-heavy circuits. Second, the parameter space is large and contains complex interdependencies—certain parameter combinations are logically invalid or lead to degenerate behavior. Third, the relationship between circuit features and optimal parameters is intricate and difficult to capture with traditional heuristics. These challenges highlight the need for a more systematic and learning-based approach that can capture the intricate relationships between circuit structure and parameter effectiveness.

Current approaches to PDR parameter tuning fall into two categories: manual tuning by experts and simple automated search methods. Manual tuning is labor-intensive, as it requires deep understanding of both the algorithm and the target domain, and does not scale to diverse verification workloads. Automated search methods, such as random sampling or grid search, fail to exploit the structural information available in the verification problem and often waste computational resources exploring invalid or suboptimal configurations [7], [8]. Furthermore, PDR engines frequently consume substantial memory resources, and solving times often exceed one hour, making such brute-force approaches impractical and unreliable. Recent advances in solver configuration [9] and circuit representation learning [10], [11] suggest that machine learning offers a promising direction for addressing these challenges effectively.

This paper presents a circuit-aware solver configuration framework that employs machine learning for intelligent heuristic selection in PDR-based verification. Our approach addresses the limitations of existing methods through three key innovations:

- **Circuit-Aware Feature Extraction:** We develop a comprehensive feature extraction pipeline that captures both graph-level structural properties of And-Inverter Graph (AIG) representations and circuit-level statistical features. This multi-level characterization enables our models to understand the verification complexity and guide parameter selection accordingly.
- **Expert-Knowledge-Based Parameter Filtering:** We incorporate human expertise through a systematic constraint-based approach to eliminate invalid parameter combinations based on logical dependencies among PDR options. This filtering mechanism significantly reduces the search space while ensuring that all considered configurations are semantically meaningful.
- **GNN-Based PDR Parameter Prediction:** We conduct a comprehensive empirical evaluation of multiple graph neural network architectures, including GraphSAGE [12], GIN [13],

GCN [14], HOGA [15] and GraphSAINT [16], to learn the complex mapping between circuit structure and optimal parameters. Through systematic comparison, we identify the most effective architecture for capturing both local node patterns and global graph properties that influence verification performance. Our approach builds upon recent advances in PDR algorithms [17]–[19] to optimize parameter selection.

Our experimental evaluation on a comprehensive benchmark suite demonstrates significant improvements over existing approaches. Our framework increases the number of solved instances by 90.0% over the default PDR configuration and by 33.3% over a commonly-used optimized setting. The system successfully identifies circuit-specific optimization patterns and generalizes well across different verification domains.

The main contributions of this work are:

- A comprehensive machine learning framework for automated PDR parameter tuning that combines circuit feature extraction, parameter filtering, and graph neural network prediction.[1]
- A systematic parameter filtering mechanism that eliminates invalid configurations based on logical dependencies, significantly reducing the search space.
- A thorough empirical evaluation of multiple GNN architectures, identifying the most effective approach for circuit-aware solver configuration.

The remainder of this paper is organized as follows: Section II provides background on PDR algorithms and parameter tuning challenges. Section III presents our circuit-aware framework including feature extraction, parameter filtering, and GNN-based prediction. Section IV describes our experimental setup and evaluation methodology, including presenting and analyzing the experimental results. Section V concludes the paper and discusses future work.

II. Preliminaries

A. Property Directed Reachability (PDR/IC3)

Property Directed Reachability (PDR) [1], also known as IC3, is a SAT-based model checking algorithm that has revolutionized the formal verification of safety properties. Unlike traditional approaches such as Bounded Model Checking (BMC) or fixed-point iteration over symbolic states, PDR constructs a sequence of inductive, relative over-approximations of reachable states in a manner directed by the property being verified. The algorithm maintains a sequence of formulas $F_0, F_1, \ldots, F_k$, where each F_i represents an over-approximation of states reachable in at most i steps from the initial states. PDR incrementally strengthens these formulas by finding and generalizing counterexamples to induction (CTIs). This incremental and goal-oriented approach has made PDR exceptionally effective, particularly in hardware verification [3].

The performance of the PDR engine in the `ABC` verification tool, however, is critically dependent on a set of heuristic parameters that control its core operations. Understanding these parameters requires deep knowledge of PDR's internal mechanisms. Algorithm 1 presents how heuristic parameters exert their influence in IC3/PDR[2]. During **Inductive Generalization** (L25-37), several parameters control how counterexamples are generalized. For instance, `-r` (TwoRound) enables a second round of literal removal for more aggressive generalization. `-g` (SkipGeneral) bypasses this expensive step entirely. `-n` (SkipDown) skips the "down" phase during clause strengthening.

[1]We will open source this work after publication.

[2]Due to space limit, we refer the readers to the prior works [1], [3], [4] for a more detailed presentation of the algorithm.

The order of generalization is managed by `-y` (FlopPrio), which uses structural flip-flop priorities to distinguish control logic from datapath, and `-f` (FlopOrder), which orders flip-flops by these priorities. For **Push Propagation** (L11-14), which determines clause propagation strategies, the `-i` (EagerPush) parameter allows pushing clauses from intermediate frames rather than only from the first frame. In **Abstraction & Refinement** (L4-7), which manages state space reduction, `-t` (UseAbs) enables abstraction by treating selected latches as pseudo-inputs, while `-k` (SimpleRefine) employs a simplified refinement strategy when abstraction is used. These parameters exhibit complex interdependencies that significantly impact PDR's behavior, and their optimal configuration depends heavily on circuit characteristics.

Algorithm 1 IC3/PDR with Heuristic Parameters in `ABC-PDR`

Require: Transition system (I, T, P), safety property P
Ensure: SAFE or counterexample trace

1: Initialize $F_0 \leftarrow I$, $k \leftarrow 0$
2: **while** true **do**
3: $k \leftarrow k + 1$; $F_k \leftarrow P$
4: **// ABSTRACTION & REFINEMENT (t: UseAbs, k: SimpleRefine)**
5: **if** t **then**
6: Apply latch abstraction
7: **end if**
8: **while** true **do**
9: $s \leftarrow$ SAT query: $F_k \wedge T \wedge \neg P'$
10: **if** s is UNSAT **then**
11: **// PUSH PROPAGATION (i: EagerPush)**
12: Push clauses between frames
13: **if** $F_j \Leftrightarrow F_{j+1}$ for some j **then return** SAFE
14: **end if**
15: **break**
16: **else**
17: $c \leftarrow$ BLOCKCTI(s, k)
18: **if** c reaches initial states **then return** counterexample
19: **end if**
20: **end if**
21: **end while**
22: **end while**

23: **procedure** BLOCKCTI(s, level)
24: $c \leftarrow$ Extract cube from state s
25: **// INDUCTIVE GENERALIZATION (g, r, n, c, y, f)**
26: **if** $\neg g$ **then** ▷ `-g`: SkipGeneral
27: $c \leftarrow$ GENERALIZE(c) with priority ordering ▷ `-y`, `-f`: FlopPrio, FlopOrder
28: **if** r **then**
29: $c \leftarrow$ GENERALIZE(c) ▷ `-r`: TwoRounds
30: **end if**
31: **if** $\neg n$ **then**
32: generalize clause with down ▷ `-n`: SkipDown
33: **end if**
34: **if** c **then**
35: generalize with CTG ▷ `-c`: Ctgs
36: **end if**
37: **end if**
38: Add $\neg c$ to frames; **return** c
39: **end procedure**

B. Graph Neural Networks

Hardware verification problems are fundamentally graph-structured, with circuits naturally represented as And-Inverter Graphs (AIGs). The structural properties of these graphs—including connectivity patterns, node types, and topological features—are intimately connected to the difficulty of verifying the circuit and the optimal configuration of verification algorithms like PDR. Traditional hand-crafted features often fail to capture the complex structural relationships that influence PDR performance, motivating the use of Graph Neural Networks (GNNs) to automatically learn meaningful circuit representations.

A GNN encodes a graph $G = (V, E)$ into a more compact representation. Starting with initial node features $\{\mathbf{x}_i^{(0)} \mid i \in V\}$, GNNs iteratively update hidden state vectors $\mathbf{x}_i^{(l)}$ using:

$$\mathbf{m}_i^{(l)} = \text{AGGREGATE}^{(l)}\left(\{\mathbf{x}_j^{(l-1)} \mid j \in \mathcal{N}(i)\}\right), \tag{1}$$

$$\mathbf{x}_i^{(l)} = \text{COMBINE}^{(l)}\left(\mathbf{m}_i^{(l)}, \mathbf{x}_i^{(l-1)}\right), \tag{2}$$

where $\mathcal{N}(i)$ are the one-hop neighbors of node i. The AGGREGATE function (e.g., mean or max pooling) pools neighboring features, and the COMBINE function integrates this with the node's current hidden state vector.

After L layers, the set of vectors $\{\mathbf{x}_i^{(L)} \mid i \in V\}$ captures the structural and feature information of the graph. A readout function then aggregates these vectors into a graph embedding for downstream tasks. For PDR parameter configuration, this enables GNNs to automatically discover which structural patterns in circuit graphs correlate with the effectiveness of different parameter settings, providing a principled approach to solver configuration that goes beyond manual heuristics.

III. Methodology: Circuit-Aware PDR Solver Configuration

A. Overview of `AutoPDR`

`AutoPDR` is a machine-learning-driven framework that automatically configures PDR solver parameters based on circuit characteristics. As shown in Figure 1, the framework addresses the fundamental challenge of PDR parameter tuning, leveraging both structural circuit features and graph neural networks to predict optimal parameter configurations for hardware model checking problems.

The core methodology of `AutoPDR` consists of three main components: ❶ **Parameter Space Filtering**, which eliminates invalid parameter combinations based on pre-defined rules; ❷ **Circuit Feature Extraction**, which captures key structural and statistical properties from the circuit's graph representation; and ❸ **Learning Circuit-Aware PDR Solver Configuration**, where a Graph Neural Network (GNN) is trained on runtime data to predict the optimal parameter configuration for a new circuit.

Unlike traditional approaches that rely on manual tuning or brute-force search, `AutoPDR` exploits the inherent structure of verification problems to guide parameter selection intelligently. The framework is designed to generalize across diverse circuit domains while maintaining computational efficiency during both training and inference phases.

B. Parameter Space Filtering

The PDR parameter space, with $2^9 = 512$ possible combinations, is too large to explore exhaustively for each input problem. A brute-force approach presents several challenges: 1) Many parameter combinations are logically invalid or counter-productive due to inherent algorithmic dependencies. 2) Evaluating all configurations each time when a new problem is encountered is computationally infeasible, even when parallel execution is employed. As suggested by Figure 3, which shows the memory consumption for one such parallel execution, running all 512 configurations for every circuit would lead to an explosion in memory and CPU core consumption. In general, a smaller and more focused parameter space is crucial for efficient model training and faster prediction.

To address these issues, `AutoPDR` incorporates a systematic filtering mechanism to prune the search space. We apply a set of implication rules derived from expert knowledge of the PDR algorithm, as summarized in Table I. These rules eliminate invalid or counter-productive configurations by formalizing dependencies between parameters.

This rule-based filtering significantly reduces the search space to 114 valid configurations, an elimination of approximately 78% of the original space. The resulted set of configurations is not only logically sound but also computationally manageable, forming a diverse and effective basis for training our prediction model.

C. Circuit Feature Extraction and Graph Representation

`AutoPDR` leverages a dual-representation approach to obtain a comprehensive understanding of each circuit, as illustrated in Figure 2. We extract high-level static features from the entire circuit structure to provide a holistic, quantitative summary. Simultaneously, we generate a detailed graph representation that is optimized for processing by a Graph Neural Network (GNN), allowing the model to learn complex topological patterns.

1) Graph Representation Preprocessing via Cone of Influence: To prepare a circuit for the GNN, we first convert its And-Inverter Graph (AIG) into a graph data structure using the `&edgelist` command, which generates both the edge list and an initial set of node features. However, industrial-scale circuits can result in massive graphs that are computationally expensive to process. To address this, we apply a *cone of influence* (COI) reduction as a crucial preprocessing step. This optimization prunes logic that does not affect the primary outputs, significantly reducing the graph's size and complexity.

Crucially, this COI reduction is applied *only* to generate the graph for the GNN. The static features, described next, are extracted from the original, unmodified circuit to ensure their accuracy. This dual approach provides the best of both worlds: complete, accurate features for high-level analysis and an efficient, optimized graph for deep learning.

To validate this preprocessing step, we conducted a preliminary analysis on the single-property AIGER benchmarks from HWMCC'13. As shown in Figure 4 and Figure 5, this study confirmed that COI reduction is both effective and efficient, eliminating an average of 16.49% of AND gates and completing in under 100ms for most instances, thus justifying its inclusion in our main experimental pipeline.

2) Static Feature Extraction: It captures both the structural complexity and statistical properties of the circuit. As depicted in Figure 2, it operates on the original AIG and encompasses the following four critical categories, chosen for its direct relevance to the PDR parameters we aim to predict:

Topological Scale and Complexity: This category quantifies the fundamental scale of the verification problem. We measure the number of primary inputs (PI), primary outputs (PO), latches (flip-flops), and AND gates. These metrics are critical for deciding on high-level trade-offs between proof search time and generalization effort. For instance, for exceptionally large circuits, the cost of the main

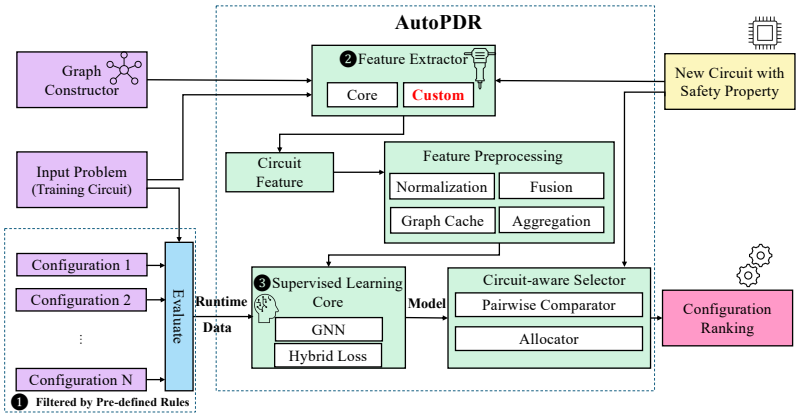


Fig. 1: Overview of AutoPDR.

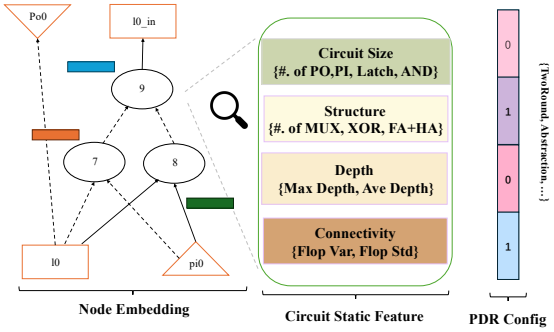


Fig. 2: Feature Extraction.

TABLE I: PDR Parameter Constraint Rules for Search Space Reduction

Parameter Rule	Algorithmic Rationale and Implementation Details
$SkipGeneral \Rightarrow \neg\{TwoRounds, SkipDown, Ctgs, FlopPrio, FlopOrder\}$	Skip generalization (`-g`) is a master switch that bypasses and nullifies several other generalization strategies, including two-round logic (`-r`), down (`-n`), CTG handling (`-c`), and structural ordering (`-y`, `-f`). This rule prevents redundant or conflicting operations when high-level generalization is disabled.
$FlopOrder \Rightarrow FlopPrio$	Flop priority (`-y`) computes structural priorities for flops. These priorities guide two different heuristics: 1) Predecessor generation, by influencing the variable selection in ternary simulation. 2) Inductive generalization, where flop ordering (`-f`) uses them to sort literals before minimization. For `-f` to be effective, `-y` must be enabled to provide meaningful priorities.
$Ctgs \Rightarrow \neg SkipDown$	Counterexample-to-generalization (`-c`) is a technique for strengthening clauses, which is only effective when the 'down' generalization strategy (`-n`) is active. If 'down' is skipped, CTG has no effect and should be disabled.
$SimpleRefine \Rightarrow UseAbs$	Simplified CEGAR refinement (`-k`) is a subordinate option that modifies the behavior of abstraction-based refinement. Therefore, it only applies when the main abstraction mechanism (`-t`) is enabled.

generalization procedure can become prohibitively high, suggesting that it should be skipped entirely (`SkipGeneral`). Furthermore, a high latch count indicates a large state space, which is the primary motivation for enabling abstraction-based techniques (`UseAbs`).

High-Level Functional Composition: This category identifies high-level functional patterns that are crucial for guiding PDR's generalization strategies. We count the number of multiplexers (MUX), XOR gates, and arithmetic units (Full and Half Adders). The presence of these functional blocks is a strong indicator that the solver may encounter complex Counterexamples-To-Generalization (CTGs) that standard literal-dropping cannot easily resolve. The "down" phase of clause strengthening is a powerful technique for handling such CTGs. Therefore, a high count of these functional structures suggests that disabling this phase (`SkipDown`) would likely be detrimental to performance.

Logic Depth and Propagation Characteristics: This category characterizes the circuit's combinational logic complexity through its maximum and average depth. These metrics are particularly relevant for PDR's frame-based reasoning. In circuits with deep logic, reaching a fixed-point inductive invariant can be slow. Enabling "eager" clause pushing (`EagerPush`) allows the solver to start propagating clauses from intermediate frames, potentially accelerating convergence. Additionally, deeper logic can complicate the abstraction-refinement loop; these features can inform the choice between standard and simplified refinement strategies (`SimpleRefine`) when abstraction (`UseAbs`) is active.

State-Space Connectivity and Influence: This category analyzes signal propagation patterns by computing the variance and standard deviation of flip-flop fanouts. These statistics capture the distribution of state variable dependencies. A high variance often indicates a clear separation between control logic (low fanout flops) and datapath logic (high fanout flops). This structural information provides a direct, quantitative basis for enabling structural flip-flop priorities (`FlopPrio`) and ordering the flip-flops by this priority during generalization (`FlopOrder`).

The extracted features are systematically normalized and encoded into an 11-dimensional feature vector $\mathbf{f}_{\text{circuit}} \in \mathbb{R}^{11}$ through a standardized transformation process. This encoding ensures consistent scaling across diverse circuit domains while preserving the essential structural characteristics that govern PDR solver performance, enabling effective machine learning-based parameter prediction.

D. Learning Circuit-Aware PDR Solver Configuration

1) Training Data Selection: The quality and composition of the data are fundamental to the model's predictive power. To ensure the model learns from meaningful and challenging instances, we first curated a dataset from the Hardware Model Checking Competition (HWMCC) benchmarks, spanning from 2007 to 2024. We specifically selected non-trivial cases where the solving time exceeded 60 seconds, filtering out instances that are too easy to provide a strong learning signal. This selection process resulted in a focused dataset comprising approximately 5% of the total available benchmarks. This dataset was then randomly partitioned into training (80%),

validation (10%), and test (10%) sets to ensure a rigorous and unbiased evaluation of the model's performance.

2) Empirical Evaluation of GNN Architecture: After establishing the circuit-aware hybrid representation, we design a GNN-based model to predict PDR solver runtime based on hardware circuit characteristics. We evaluated several prominent GNN models and use Kendall's τ and Spearman's ρ as evaluation metrics, which measure the ordinal association between predicted and actual rankings of PDR parameter configurations for each hardware circuit in model checking tasks:

$$\tau = \frac{N_c - N_d}{n(n-1)/2}, \quad \rho = 1 - \frac{6\sum d_i^2}{n(n^2-1)} \tag{3}$$

where N_c and N_d are concordant and discordant pairs, and d_i is the rank difference for each observation.

Table II shows the performance of various GNN encoders on the held-out test set. **GraphSAGE** emerges as the top-performing architecture, achieving the highest Kendall's τ (0.508) and Spearman's ρ (0.683). Its inductive learning capability, which allows it to generalize to unseen hardware circuit graphs, and its neighborhood aggregation mechanism are particularly well-suited for capturing the circuit-specific structural and topological features that influence PDR solver performance on different hardware verification problems.

TABLE II: Performance comparison for GNN encoders on the test set. BiGraphSAGE uses two separate SAGE modules to aggregate from predecessors and successors in directed graphs and then combines their embeddings.

Encoder	Kendall's τ	Spearman's ρ
GraphSAGE [12]	**0.508**	**0.683**
Bi-GraphSAGE	0.432	0.640
GraphSAINT [16]	0.446	0.594
GCN [14]	0.348	0.512
GIN [13]	0.390	0.520
HOGA [15]	0.484	0.583

The model first computes node-level embeddings using the GraphSAGE encoder, which are then aggregated into a single circuit topology embedding, $\mathbf{h}_{\text{circuit}}$, via a global mean pooling operation. This embedding is then concatenated with the verification-oriented feature vector $\mathbf{f}_{\text{circuit}}$ and the target PDR parameter vector $\mathbf{p}_{\text{pdr}}$:

$$\mathbf{h}_{\text{combined}} = \text{concat}(\mathbf{h}_{\text{circuit}}, \mathbf{f}_{\text{circuit}}, \mathbf{p}_{\text{pdr}}) \tag{4}$$

This combined vector, which encapsulates hardware circuit features and model checking information, is fed into a multi-layer perceptron

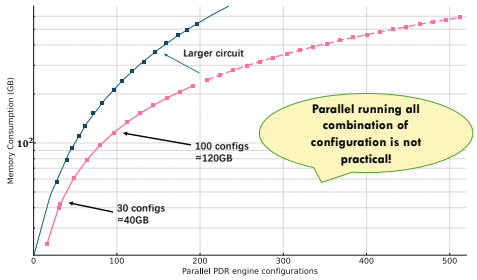


Fig. 3: Memory consumption of parallel execution of PDR engines for a small and a large verification problem. The pink line represents the small case `6s0` with 157 state bits from the HWMCC13 benchmark. The blue line represents `ILA_Flute_SUB_problem` with 174263 state bits from the HWMCC24 benchmark.

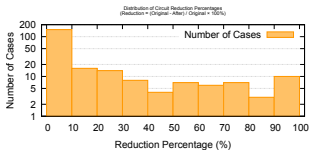


Fig. 4: Distribution of COI reduction percentages across 225 benchmarks from HWMCC'13.

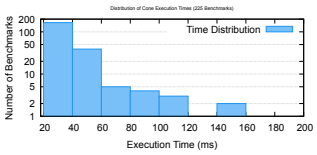


Fig. 5: Distribution of COI execution times across the same 225 benchmarks.

(MLP) to predict the final PDR solver runtime for the specific circuit-parameter combination. Figure 6 shows the relative importance of the verification-oriented circuit features, confirming that our selected hardware circuit properties are highly relevant to the PDR solver configuration prediction task.

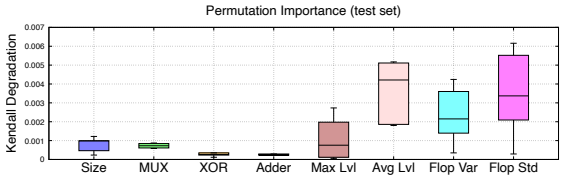


Fig. 6: Permutation test for feature importance. The y-axis shows the degradation in Kendall's τ when a feature is permuted; higher values indicate greater importance.

Figure 6 illustrates the relative importance of different verification-oriented hardware circuit features, as determined by a permutation test. The test measures the degradation in Kendall's τ when the values of a single circuit feature are randomly shuffled. A larger degradation indicates higher importance for hardware circuit-specific PDR solver configuration prediction. The results show that **Flop Std** (standard deviation of flip-flop fanout) and **Avg Lvl** (average logic level) are the most influential hardware circuit characteristics. This aligns with our model checking domain knowledge, as flip-flop connectivity patterns and combinational logic depth are known to critically impact PDR solver behavior on different hardware verification problems.

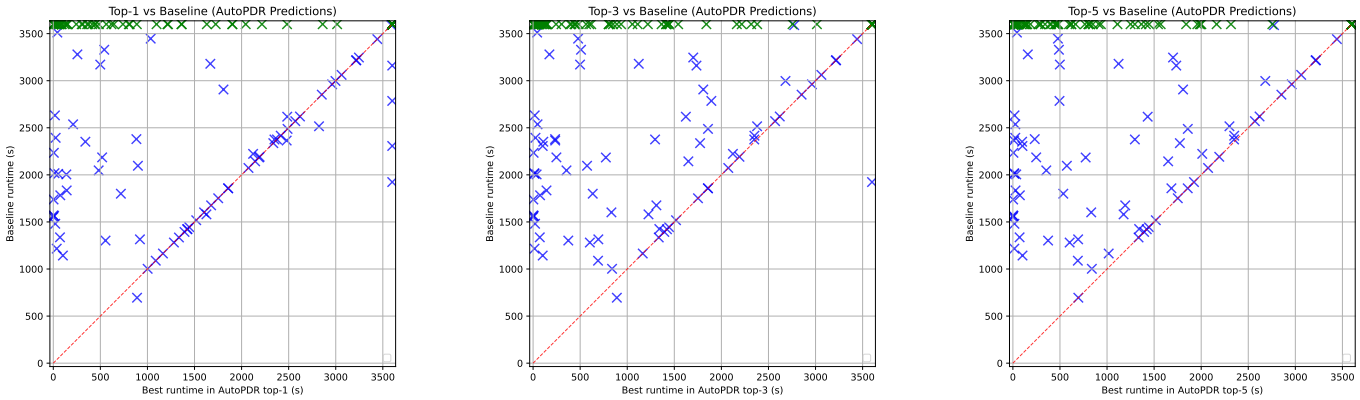
3) Rank-Aware Hybrid Loss Function: A key insight of our work is that precisely predicting the absolute runtime of a PDR configuration on a specific circuit is a secondary goal. The primary objective for practical application is to **correctly rank the configurations** for each circuit to identify the best one for that particular circuit. A standard Mean Squared Error (MSE) loss is insufficient as it equally penalizes all prediction errors and does not explicitly optimize for circuit-specific ranking quality.

To address this, we introduce a **Rank-Aware Hybrid Loss Function** that aligns the training objective with the ultimate goal of ranking. This loss function combines a regression component with a pairwise ranking component:

$$\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{MSE}} + (1-\alpha) \cdot \mathcal{L}_{\text{ranking}} \tag{5}$$

where α is a balancing hyperparameter, which we set to 0.3.

The regression component, $\mathcal{L}_{\text{MSE}}$, is computed on the log-transformed runtimes to handle their wide dynamic range and focus on relative error. The ranking component, $\mathcal{L}_{\text{ranking}}$, is a pairwise hinge loss that directly penalizes incorrectly ordered pairs of configurations. For any two configurations i and j in a batch, where the true runtime



(a) Top-1 predicted config. vs baseline *pdr*

(b) Top-3 predicted config. vs baseline *pdr*

(c) Top-5 predicted config. vs baseline *pdr*

Fig. 7: Performance comparison between AutoPDR's Top-k predicted configurations and the default configuration. When $k > 1$, the shortest runtime among the Top-k configurations is used. Points above the diagonal line indicate cases where our predicted configurations outperform the baseline *pdr*.

$t_i < t_j$, the loss is formulated as:

$$\mathcal{L}_{\text{ranking}} = \frac{1}{|\mathcal{P}|} \sum_{(i,j)\in\mathcal{P}} \max(0, m - (\hat{t}_j - \hat{t}_i)) \tag{6}$$

where $\mathcal{P}$ is the set of all pairs where $t_i < t_j$, $\hat{t}$ are the predicted runtimes, and m is a margin hyperparameter. This loss encourages the model to predict a lower runtime for configuration i than for j, thus preserving their relative order.

By optimizing this hybrid objective, AutoPDR learns not only to estimate runtimes but, more importantly, to discern the subtle performance differences between parameter sets for specific circuits, making it highly effective at its core task of finding the optimal configuration for each individual circuit.

IV. Experiments

A. Experimental Setup

Our experimental setup is designed to rigorously evaluate the performance and generalization capabilities of AutoPDR. The framework is implemented in Python using PyTorch and PyTorch Geometric, with circuit feature extraction and graph construction accelerated via a Rust interface to ABC [20]. All experiments were conducted on a Ubuntu 20.04.4 LTS server with Intel Xeon Platinum 8375C CPUs, an NVIDIA 3090 GPU, and 128GB of memory, with a one-hour wall-clock time limit per task.

To assess the generalization capability of our method, we carefully curated a challenging evaluation benchmark of 192 circuits from HWMCC07 to HWMCC24 under the following criteria:

- **Nontrivial and unsolved problems:** To test the framework's ability to handle difficult cases, we included instances where the baseline *pdr* configuration exceeded a 900-second solve time, many of which were previously unsolved.
- **Circuits with unseen properties:** To evaluate generalization across different verification tasks on the same circuit, we included instances that may share the name with a circuit in the training set but target a different property.
- **Unseen circuits:** To assess broader generalization, we included circuits that may share the same family name as the training data (e.g., '6s', 'intel') but with distinct circuit structures.

These selection criteria ensure that our evaluation is conducted on a diverse and challenging set of problems not seen during training.

TABLE III: Performance Comparison with Baseline Configurations

Method	Safe	Unsafe	Timeout	Solved	Improvement
Baseline (*pdr*)	65	15	112	80	Baseline
Baseline (*pdr-nct*)	82	32	78	114	+34 (42.5%)
AutoPDR Top-1	**94**	27	71	**121**	**+41 (51.2%)**
AutoPDR Top-3	**109**	31	52	**140**	**+60 (75.0%)**
AutoPDR Top-5	**117**	**35**	40	**152**	**+72 (90.0%)**

B. Performance Analysis

We analyze the performance of AutoPDR by comparing its Top-k predicted configurations against two baselines: the default *pdr* configuration and a commonly-used optimized setting, *pdr-nct*.

Table III summarizes the results, showing that AutoPDR significantly improves solving capability. The Top-5 configurations achieve the highest performance, solving 152 out of 192 cases (79.2% success rate) compared to the default's 80 solved cases (41.7%). This represents a 90.0% increase in solved instances. The performance progressively improves from Top-1 (121 solved) to Top-3 (140 solved) and Top-5 (152 solved), demonstrating that providing multiple diverse configurations enhances the likelihood of finding an optimal parameter set.

Figure 7 provides a visual comparison between AutoPDR's Top-k predictions and the default baseline. Each scatter plot compares the runtime of a predicted configuration (x-axis) against the baseline runtime (y-axis). Points above the diagonal indicate instances where AutoPDR's prediction is superior. A significant number of points fall in this region, particularly for Top-3 and Top-5 predictions, highlighting consistent performance gains. The plots also distinguish between cases where the baseline solved the instance (blue) and where it timed out (green), underscoring our approach's effectiveness on challenging verification tasks.

V. Conclusion

This paper introduced AutoPDR, a novel machine learning framework for automated PDR parameter tuning. By combining systematic parameter filtering with a GNN-based model that learns from both static and graph-based circuit features, our approach effectively navigates the complex PDR parameter space. Our experimental evaluation

demonstrated that `AutoPDR` significantly outperforms default configurations, increasing the number of solved instances by 90.0%. The framework successfully identifies circuit-specific parameter patterns and generalizes to unseen problems, making it a valuable tool for automated formal verification workflows. Future work will focus on extending the framework to a wider range of parameters and exploring dynamic, reinforcement learning-based adaptation.

REFERENCES

[1] A. Bradley, "SAT-based model checking without unrolling," in *International Workshop on Verification, Model Checking, and Abstract Interpretation*. Springer Berlin Heidelberg, 2011, pp. 70–87.

[2] K. Hoder and N. Bjørner, "Generalized property directed reachability," in *International Conference on Computer Aided Verification*. Springer, 2012, pp. 157–171.

[3] N. Eén, A. Mishchenko, and R. Brayton, "Efficient implementation of property directed reachability," in *Formal Methods in Computer-Aided Design*, 2011, pp. 125–134.

[4] A. Griggio and M. Roveri, "Comparing different variants of the IC3 algorithm for hardware model checking," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 6, pp. 1026–1039, 2015.

[5] G. Cabodi, P. E. Camurati, A. Mishchenko, M. Palena, and P. Pasini, "SAT solver management strategies in IC3: an experimental approach," *Formal Methods in System Design*, vol. 50, pp. 39–74, 2017.

[6] H. H. Hoos, *Automated algorithm configuration and parameter tuning*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 37–71.

[7] F. Hutter, M. Lindauer, A. Balint, S. Bayless, H. Hoos, and K. Leyton-Brown, "The configurable sat solver challenge (cssc)," *Artificial Intelligence*, 2016.

[8] A.-M. Leventi-Peetz, J.-V. Peetz, and M. Rohde, "Ml supported predictions for sat solvers performance," *arXiv preprint arXiv:2112.09438*, 2021.

[9] H. Wu *et al.*, "Cubing for tuning," *arXiv preprint arXiv:2504.19039*, 2025.

[10] ——, "Circuit representation learning with masked gate modeling and Verilog-AIG alignment," *arXiv preprint arXiv:2502.12732*, 2025.

[11] C. Yu *et al.*, "BoolGebra: Attributed graph-learning for boolean algebraic manipulation," *arXiv preprint arXiv:2401.10753*, 2024.

[12] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems*, 2017.

[13] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" *arXiv preprint arXiv:1810.00826*, 2018.

[14] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations*, 2017.

[15] C. Deng, Z. Yue, C. Yu, G. Sarar, R. Carey, R. Jain, and Z. Zhang, "Less is more: Hop-wise graph attention for scalable and generalizable learning on circuits," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[16] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. Prasanna, "Graph-saint: Graph sampling based inductive learning method," *arXiv preprint arXiv:1907.04931*, 2019.

[17] T. Seufert, C. Scholl, A. Chandrasekharan, S. Reimer, and T. Welp, "Making PROGRESS in property directed reachability," in *International Conference on Verification, Model Checking, and Abstract Interpretation*. Cham: Springer International Publishing, 2022, pp. 355–377.

[18] M. Kori, F. Ascari, F. Bonchi, R. Bruni, R. Gori, and I. Hasuo, "Exploiting adjoints in property directed reachability analysis," in *International Conference on Computer Aided Verification*. Cham: Springer Nature Switzerland, 2023, pp. 41–63.

[19] M. Blankestijn and A. Laarman, "Incremental property directed reachability," in *International Conference on Formal Engineering Methods*. Singapore: Springer Nature Singapore, 2023, pp. 208–227.

[20] R. Brayton and A. Mishchenko, "ABC: An academic industrial-strength verification tool," in *Computer Aided Verification: 22nd International Conference, CAV 2010, Edinburgh, UK, July 15-19, 2010. Proceedings 22*. Springer, 2010, pp. 24–40.